

Into the Backend

Compiler Engineering #5: Target constructs, Machine IR, and the machine pipeline

Contents

- 1. Session Bridge 2
- 2. Chapter 9: Connecting a Target 5
- 3. Chapter 11: Machine IR 10
- 4. Chapter 12: The MC Layer 25
- 5. Chapter 13: The Machine Pass Pipeline 30

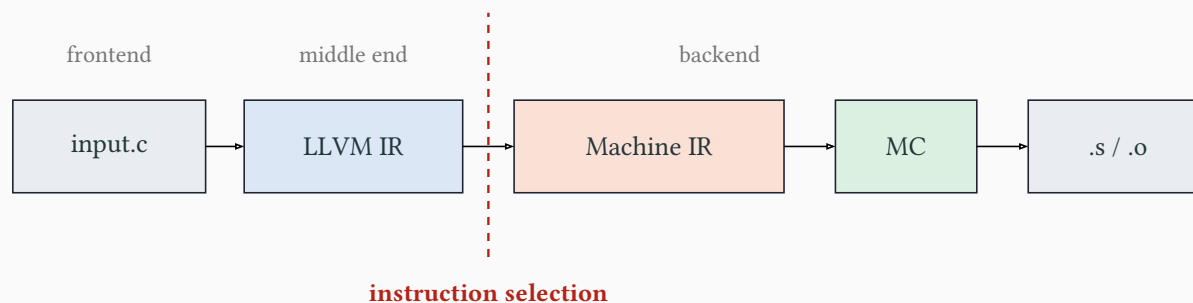
1. Session Bridge

Where We Are

So far, we stayed in the middle end: how to build LLVM, what IR **is**, how to write a pass, and what passes already exist.

Session 4 ended on a cliff-hanger. Every interesting pass asked the **target** something:

```
LoopVectorize      -> TargetTransformInfo::getRegisterBitWidth
SimplifyLibCalls   -> TargetLibraryInfo
CodeGenPrepare     -> TargetLowering::isLegalAddressingMode
```



Today: we cross the red line.

Chapter	What we take from it
9	Connecting a target: intrinsics, <code>TargetTransformInfo</code> , injecting passes
11	Machine IR: the <code>.mir</code> format, operands, registers, SSA-ness
12	The MC layer — assembly syntax and encodings (lightly)
13	The machine pass pipeline: three stages, and where to inject

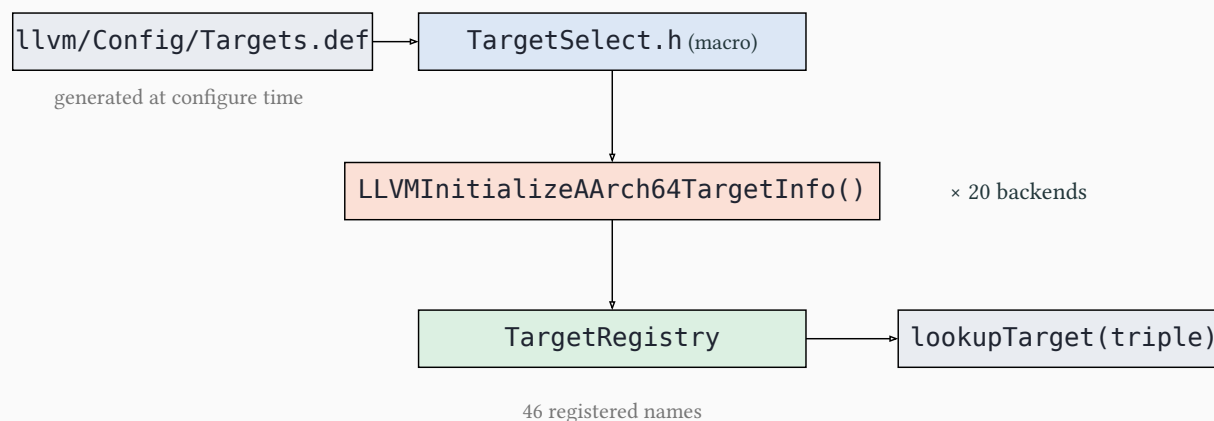
2. Chapter 9: Connecting a Target

A Backend Is Six Directories

directory	library	holds	ch.
TargetInfo/	LLVMAArch64Info	the Target singleton	9
MCTargetDesc/	LLVMAArch64Desc	the MC layer	9, 12
<root>/	LLVMAArch64CodeGen	TargetMachine, ISel, .td	9, 11, 13
AsmParser/	LLVMAArch64AsmParser	.s → MCInst	12
Disassembler/	LLVMAArch64Disassembler	.o → MCInst	—
Utils/	LLVMAArch64Utils	shared helpers	—

Each directory becomes **one library**. `llvm-config --components` lists them, so you can read a real backend's shape without a source checkout.

Who Calls LLVMInitialize*?



The chapter says the build system “automatically populates `InitializeAllTargets`”. It is literally a macro over a generated list — `Targets.def` is written at configure time, and `TargetSelect.h` expands it once to **declare** your three functions and once to **call** them.

That is why they must be `extern "C"`: these are generated call sites resolved by **symbol name**. C++ mangling breaks the link.

Intrinsics: The Case Against

The chapter's own example intrinsic — a widening $16 \times 16 \rightarrow 32$ multiply — **does not need to be an intrinsic**:

```
int widening_smul(short a, short b) { return a * b; }
```

C promotes both operands to `int`. Plain C already said it. Therefore, this intrinsic is not useful. (But it is the best for intro)

*With no target, the vectorizer does **nothing at all**.*

The default `TargetTransformInfo` reports no vector registers, so vectorising is never profitable.

A backend that has not implemented TTI is not “using sensible defaults”.
It is telling every cost-driven pass in the middle end that everything is equally cheap.

3. Chapter 11: Machine IR

Why a Second IR

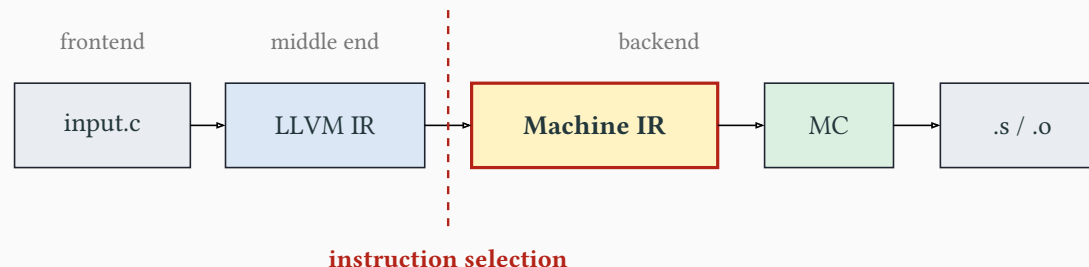
“the LLVM IR isn’t flexible enough”

Specifically: **SSA**. Real architectures are not in SSA form, and SSA is so deep in LLVM IR’s design that abandoning it would mean redesigning the Core library’s API.

Machine IR gives it up on purpose:

- supports SSA **and** non-SSA
- very few structural rules — a **MachineBasicBlock** may have **zero or several** terminators, where an LLVM IR block needs exactly one
- extends through TableGen

The price: an API that is hard to approach, because almost nothing is checked for you.



The .mir Format

YAML. One LLVM IR section, then one section per `MachineFunction`.

```
--- |
  ; textual LLVM IR
...
---
name:      funcName
tracksRegLiveness: true
body:     |
  bb.0 (%ir-block.entry):
    successors: %bb.1(0x80000000)
    liveins: $x0, $x1
    %0:gpr64 = COPY $x0
...
```

One instruction per line: `def0, def1 = opcode arg0, arg1 :: mem ops`, with `%` virtual, `$` physical, `@` symbol, bare number immediate.

Shrinking Is The Skill

Step	Lines
<code>llc -stop-before=peephole-opt</code>	114
<code>+ -simplify-mir</code> — drops defaulted and recomputable fields	75
by hand — no IR section, no <code>registers</code> / <code>frameInfo</code> / <code>liveins</code> / <code>successors</code>	27

Only `name` and `tracksRegLiveness` are mandatory.

The test of whether you shrank it legally is that it still runs:

```
$ llc -mtriple=aarch64 -run-pass=peephole-opt shrunk.mir -o -  
--- |  
define void @def_in_loop_use_outside() {  
entry:  
    unreachable          # <- the parser re-synthesised this
```

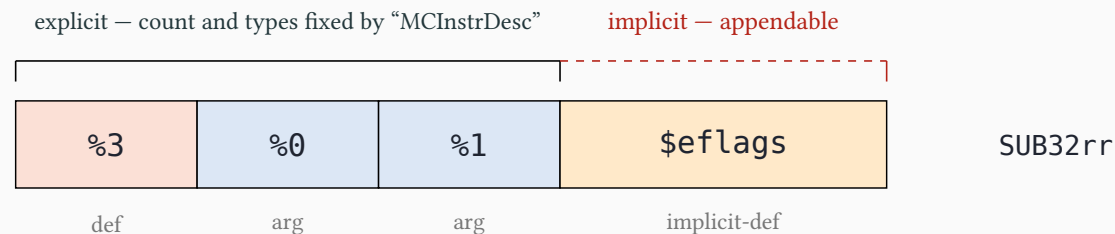
Four Traps When Shrinking

Delete	What actually happens
successors:	edges survive (rebuilt from terminators), probabilities do not — they become uniform. Fatal only for profile-driven passes. Best compromise: keep it without the (0x...) weights
liveins:	it appears at two nesting levels. Top-level: near-useless. Inside body: : real dataflow — delete it and incoming arguments look dead
the IR section	the triple went with it. -mtriple must come back... and on a matching host, omitting it silently works
...with a call in it	error: use of undefined global value '@sink' . The book's one-line fix needs two

Plus the one that is not about shrinking: **.mir** does not serialise **analysis state**. Chaining **-run-pass=opt1** then **-start-after=opt1** recomputes everything in between, so a stale-analysis bug vanishes.

MachineInstr: One Class, All Targets

No subclass hierarchy. **Every** instruction on **every** target is a `MachineInstr`. Identity comes from the opcode.



One `MCInstrDesc` per opcode, immutable: the properties (`mayLoad`, `isCommutable`), the **statically expected** operand count, their types and constraints. Violate it and `MachineVerifier` rejects the instruction.

Consequence: **“is this an add?” is not a target-independent question.** To ask it generically you must expose a property and implement the hook — the model is `isRegSequenceLike` + `TargetInstrInfo::getRegSequenceInputs`.

Implicit Operands, Flavour 1: Hardware

`@sel` in C is a compare and a select. On x86:

```
%3:gr32 = SUB32rr  %0, %1,      implicit-def $eflags
%4:gr32 = CMOV32rr %1, %0, 15,  implicit  $eflags
%5:gr32 = ADD32rr  %4, %2,      implicit-def dead $eflags
```

The compare **writes** the flags; the conditional move **reads** them. Neither is expressible in the C source — the hardware gives you no choice about the destination. **dead** on the third says nobody reads that write.

`MCInstrDesc` guarantees the order, which is why indexing works at all:

1. explicit **definitions** (registers only) — printed left of the `=`
2. explicit **arguments**, in description order
3. **implicit** operands, defs and uses mixed, **unordered**

Implicit Operands, Flavour 2: The ABI

```
CALL64pcrel32 @bar, csr_64, implicit $rsp, implicit $edi,  
             implicit-def $rsp, implicit-def $eax
```

Two explicit operands. This instruction defines nothing explicitly.

@bar	the symbol — the only thing the instruction really takes
csr_64	a register mask : every register in it is preserved across the call. The callee-saved set, compactly. Absence from the mask means clobbered
implicit \$edi	the argument. Implicit because the compiler imposes it, not the hardware
implicit-def \$eax	the return value

The final assembly is `call bar`. None of the rest appears.

*This is why implicit operands are not all statically known: **MCInstrDesc** fixes the explicit ones, but you may **append** implicit operands. Beware — a **COPY** is expected to have exactly two operands, and adding more makes the copy optimiser give up.*

Tied Operands Break SSA – On Purpose

x86 arithmetic is two-address, and the `twoaddressinstruction` pass is what introduces the constraint:

after isel

```
%4:gr32 = CMOV32rr %1, %0, 15  
          (three distinct regs)
```

after two-address

```
%4:gr32 = COPY killed %1  
%4:gr32 = CMOV32rr %4, killed %0, 15
```

`%4` is operand 0 (a **def**) and operand 1 (a **use**) — the model for `a = cond ? a : b`. It tells the register allocator both must get the same physical register.

And look at the cost: `%4` **now has two definitions**.

Tied operands are **incompatible with SSA**.

Which is exactly why this pass runs where it does — after PHI elimination.

Early Clobber Is One Character

```
"=&r"  -> regdef-ec:GPR64common, def early-clobber %2  
"=r"   -> regdef:GPR64common,   def %2
```

early-clobber says the definition is produced **before** the arguments are read — so it must not share a register with them.

Necessary whenever one “instruction” is really several:

```
asm("mov $0, $1\n add $0, $0, $2", "=&r,r,r"(b, c));
```

To the register allocator this whole block is **one** instruction. Without **&**, **\$0** could be assigned **\$1**’s register — and the **add** would read a value the **mov** already destroyed.

*The default expectation is the opposite: definitions are produced **after** all arguments are read. Early clobber is how you say otherwise.*

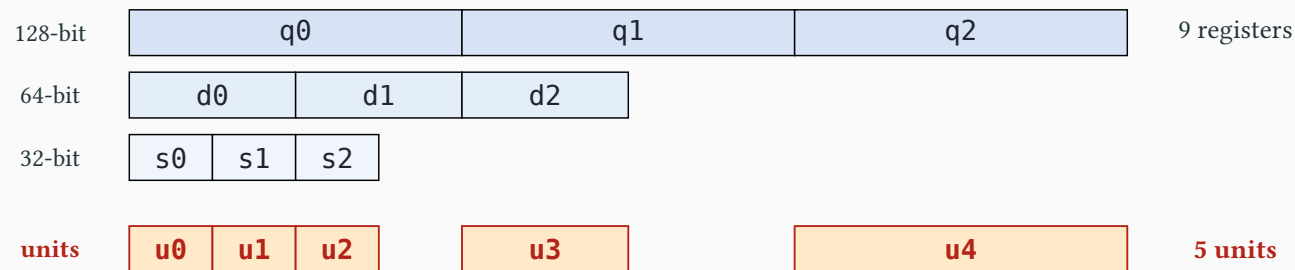
Registers: Four Concepts

Register class	a set sharing a property (32-bit, FP-capable). A register belongs to zero or more
Sub-registers	real hierarchy. x86 <code>ax</code> contains <code>al</code> , <code>ah</code> ; writing one writes the parent. Register aliasing . Exposed as <code>%1.sub_32</code>
Register tuples	logical super-registers. Hardware has only 32-bit regs, but you want pairs <code>(r0, r1)</code> , <code>(r1, r2)</code> → model <code>r0_r1</code> . The API does not distinguish logical from real
Register units	the abstraction that keeps the above affordable

Naming convention worth internalising: **Machine*** is **per-function and dynamic**; **Target*** is **per-subtarget and static**. `MachineRegisterInfo` is about the function being compiled; `TargetRegisterInfo` about the architecture.

Register Units, Measured

Figure 11.2 — nine registers, three **irregular** layers. Empty cells are not addressable:



The bottom row is what `llvm-tblgen -gen-register-info` computes. **Five units cover nine registers**. “Is **q0** free?” becomes 3 unit checks instead of walking `d0, d1, s0, s1, s2`.

```
$ lab/run.sh registers
```

...And A Number The Book Does Not Mention

We declared **three** register classes. TableGen emitted **seven**:

```
SINGLES DOUBLES DOUBLES_with_sub32_low DOUBLES_with_sub32_high
QUADS   QUADS_with_sub64_low   QUADS_with_sub32_high
```

Because the hierarchy is **irregular** — **d2** has no singles beneath it — “a double” and “a double whose low half is addressable” are genuinely different operand constraints.

Nobody asked for these. The **shape of the register file** demanded them.

Exercise 2 measures three follow-ups:

Edit	regs	classes	units
baseline	10	7	5
drop CoveredBySubRegs	10	7	5
regularise (add s3)	11	6	6
add a tuple	11	8	5

The last row is the entire argument for register units: a new register, a new class, **zero** new units.

SSA at the Machine Level – Quiz 1

The rule, exactly: **the Machine IR is in SSA form so long as every virtual register is defined at most once.**

Assume `dsub_x` are **non-overlapping** quarters. Is this SSA?

```
undef %4.dsub_0:dquad = COPY %0
    %4.dsub_1:dquad = COPY %1
    %4.dsub_2:dquad = COPY %2
    %4.dsub_3:dquad = COPY %3
```

No. The unit of the rule is the **virtual register**, not the lane. `%4` is on the left of `=` four times.

“But they write different bits” is a **liveness** argument. Different question.

The SSA form is **REG_SEQUENCE**, which builds the whole register in **one** definition — and carries no encoding, so it must die before MC.

Two Conventions To Respect

While in SSA form, the code generator maintains:

1. PHI instructions exist only in SSA form.
2. Physical registers have the shortest live range possible.

Don't

```
$r0 = opcl ...  
...  
... = opcN $r0
```

Do

```
%v0 = opcl ...  
...  
$r0 = COPY %v0  
... = opcN $r0
```

On the left, `$r0` is live across the whole region: the allocator cannot use it for anything else, and `opcl` is pinned to a physical register so optimisations give up on it. On the right, both problems are one `COPY` wide.

*Generic passes **assume** both. Break them and the failures look like the optimiser being mysteriously bad.*

4. Chapter 12: The MC Layer

Why Now, Not Later

MC is the **last** stage of the backend — `MCInst` on the way to assembly or an object file. The book visits it early for one reason:

*ISAs group instructions into **encoding families**,
and your `TableGen` classes should mirror that grouping.*

Discover it late and you rewrite every instruction record.

Chapter 11 gave records their operand lists. Chapter 12 adds two fields:

<code>AsmString</code>	the syntax, binding <code>\$name</code> operands. A <code>printf</code> format whose slots are filled by each operand type's <code>PrintMethod</code>
<code>Inst</code>	the encoding, as <code>bits<N></code> assigned range by range. Not declared in the <code>Instruction</code> class — the generator just looks for it

40 Lines of .td, and the Encoder Is Written



tiny.td

```
class ALU<bits<4> opc, string m>
  : Instruction<> {
    bits<16> Inst;
    bits<2> dst;
    let Inst{15-12} = opc;
    let Inst{11-10} = dst;
    let Inst{9-8}   = src0;
    let Inst{7-6}   = src1;
    let AsmString = m # " $dst, ...";
  }
def ADD : ALU<0b0001, "add">;
```

gen-emitter output

```
static const uint64_t InstBits[] = {
    UINT64_C(4096), // ADD
    UINT64_C(8192), // SUB
};
case Tiny::ADD:
    // op: dst
    Value |= (op & 0x3) << 10;
    // op: src0
    Value |= (op & 0x3) << 8;
    // op: src1
    Value |= (op & 0x3) << 6;
```

No C++ was written. **The declarative bit positions are the encoder.**

`4096 = 0b0001 << 12`. The opcode arrives pre-shifted.

```
$ lab/run.sh encoding
```

The Same Round Trip, On A Real ISA

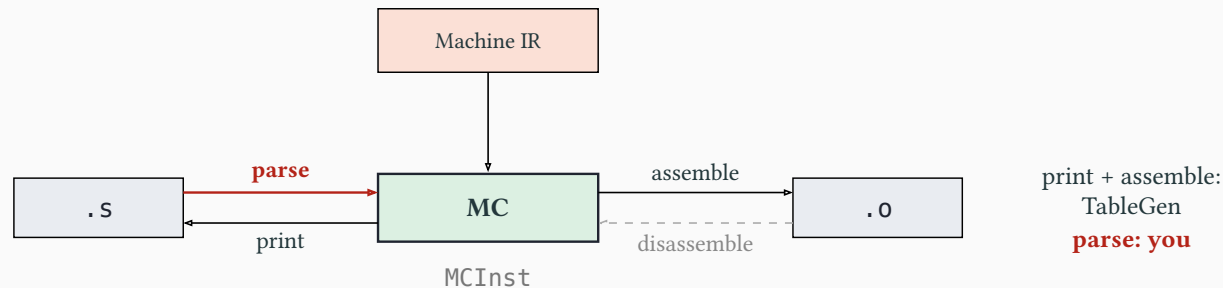
```
$ llvm-mc -triple=aarch64 --show-encoding --show-inst add.s
add w0, w1, w2    // encoding: [0x20,0x00,0x02,0x0b]
                  // <MCInst #1729 ADDWrs
                  // <MCOperand Reg:W0> <MCOperand Reg:W1>
                  // <MCOperand Reg:W2> <MCOperand Imm:0>>
```

Four operands where the assembly wrote three. `ADDWrs` is the shifted-register form; the shift amount is a real operand holding 0. The mnemonic hides it; `MCInstrDesc` does not.

```
0x0b020020 = 0b00001011000000100000000000100000
sf [31]      = 0    32-bit operands -> the W registers
Rm [20:16]   = 2    -> w2
Rn [ 9: 5]   = 1    -> w1
Rd [ 4: 0]   = 0    -> w0
```

AArch64's own `.td` wrote those ranges the same way `tiny.td` does.

Three Components, Unequally Generated



Every binary tool is a path through MC. An assembler is **parse** then **assemble**; a disassembler is the reverse. **MCInstPrinter** and **MCCodeEmitter** come from TableGen — **MCTargetAsmParser** is mostly yours.

The asymmetry is visible in the diagnostics: hand-written, and they know the register classes.

```
$ llvm-mc -triple=aarch64 bad.s      # add w0, w1, x2
bad.s:1:13: error: expected compatible register, symbol
           or integer in range [0, 4095]
```

Bit order is the trap (quiz 2): write ranges high-to-low, **Inst{3-0}**, as ISA documents draw them. **Inst{0-3}** is legal TableGen and **reversed**.

*Not every instruction needs an encoding — only those reaching MC. **PHI** and **COPY** carry **isPseudo** and have none. That is quiz 3.*

5. Chapter 13: The Machine Pass Pipeline

Three Stages

	aarch64-isel	phi-node-elimination	greedy / virtregrewriter	asm-printer
isSSA	true	false	false	
noPhis	false	true	true	
noVRegs	false	false	true	
	SSA + vreg	vreg only	physreg only	

```
%1:gpr64sp = PHI %6, %bb.0, %2, %bb.1          (1)
%8:gpr64common = ADDXri %1, 1, 0

%1:gpr64sp = COPY killed %12                  (2)
%12:gpr64sp = COPY killed %2    <- %12 also def'd in bb.0

renamable $x8 = ADDXri killed renamable $x8, 1, 0  (3)
```

How SSA ends: PHI elimination puts a **COPY** in each predecessor. One copy per predecessor → the destination gets **two** definitions. Same dataflow, no longer SSA.

```
$ lab/run.sh pipeline
```

Where To Inject

`TargetPassConfig::addMachinePasses` builds the default pipeline.

Do not override it.

Its hooks — `addPreISel`, `addPreRegAlloc`, `addPreEmitPass`, ... — **are** the injection mechanism. Override the parent and you lose all of them, plus every generic optimisation it installs.

One correction chapter 13 makes to chapter 9: to inject **LLVM IR** passes, prefer `addPreISel` over `addIRPasses`. Overriding `addIRPasses` drops the default IR passes — chapter 9's example only works because it calls the parent first.

```
$ llc -mtriple=aarch64 -O1 -debug-pass=Structure loop.ll
aarch64-isel finalize-isel phi-node-elimination
twoaddressinstruction register-coalescer machine-scheduler
virtregmap greedy virtregrewriter prologepilog
postrapseudos aarch64-asm-printer
```

30 distinct `aarch64-*` passes appear in that listing. The default pipeline is a backbone; the target fills it in.

Reusing a Generic Pass: The Recipe

Chapter 13 does not survey the Machine passes — most are the Machine-level twin of an LLVM IR pass you know (`MachineLICM` → `LICMPass`). It teaches a **procedure** instead:

1. **Find it.** `llvm/include/llvm/CodeGen/Passes.h` lists every `createXXXPass` and `XXXPassID`. (Dead code elimination = `DeadMachineInstructionElimID`.)
2. **Place it.** Pick the `addXXX` hook, call `addPass` with the ID.
3. **Does nothing?** Reduce a `.mir`, iterate with `-run-pass`, read the debug log.
4. **Log not enough?** Read the pass. Look for `TRI` / `TII` / `TLI` uses, and for `MachineInstr::isXXX` calls (grep `->is` and `.is` near a variable named `MI`).

Step 4 is the transferable skill:

a generic Machine pass is a **template with target-shaped holes**,
and reading it tells you which holes are load-bearing.